

Theory Assignment-2

1.What is CSS Box Model?

A. The **CSS Box Model** describes how every HTML element is treated as a box.

It has **4 parts**:

1. **Content** → actual text/image
2. **Padding** → space between content and border
3. **Border** → boundary around the element
4. **Margin** → space outside the border

Example:

```
div {  
  
width: 200px;  
  
padding: 20px;  
  
border: 2px solid black;  
  
margin: 10px;  
  
}
```

2.What is Media Query?

A. A **media query** is used to make a webpage **responsive**.

It allows CSS to change according to the **screen size/device**.

Example:

```
<!DOCTYPE html>  
<html>  
<head>  
  <style>  
    body {  
      background-color: lightblue;  
    }  
    @media (max-width: 600px) {  
      body {  
        background-color: pink;  
      }  
    }  
  </style>
```

```
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

3. What are the features of Bootstrap?

Bootstrap is a popular CSS framework used to create responsive websites quickly.

Main features:

- **Responsive design**
- **Grid system**
- Ready-made **buttons**
- Ready-made **forms**
- Ready-made **cards**
- **Navbar**
- **Alerts**
- **Tables**
- Utility classes
- Mobile-first design

Example:

```
<button class="btn btn-primary">
  Click Me
</button>
```

4. Basic Grid Structure in Bootstrap

Bootstrap uses a **12-column grid system**.

Basic structure:

```
<div class="container">
  <div class="row">
    <div class="col-4">
```

```
    Column 1
</div>

<div class="col-4">
    Column 2
</div>

<div class="col-4">
    Column 3
</div>

</div>

</div>
```

5. What are CSS Selectors?

CSS selectors are used to **select HTML elements** that we want to style.

1. Element selector

```
p{
    color: red;
}
```

Selects all <p> elements.

2. ID selector

```
#heading{
    color: blue;
}
```

HTML:

```
<h1 id="heading">Hello</h1>
```

3. Class selector

```
.box{
    background-color: yellow;
}
```

HTML:

```
<div class="box">Hello</div>
```

4. Universal selector

```
* {  
    margin: 0;  
}
```

Selects all elements.

5. Group selector

```
h1, p {  
    color: green;  
}
```

6. What are CSS Backgrounds? List the properties.

CSS background properties are used to **style the background of an element**.

Important properties:

- background-color
- background-image
- background-repeat
- background-position
- background-size
- background-attachment

Example:

```
body {  
    background-color: lightblue;  
    background-image: url("image.jpg");  
    background-repeat: no-repeat;  
    background-position: center;  
    background-size: cover;  
}
```

7. How can we overlap elements in CSS?

We can overlap elements using:

position + z-index

Example:

```
<div class="box1">Box 1</div>
```

```
<div class="box2">Box 2</div>
```

```
.box1 {  
    position: absolute;  
    top: 50px;  
    left: 50px;  
    width: 100px;  
    height: 100px;  
    background: red;  
}
```

```
.box2 {  
    position: absolute;  
    top: 80px;  
    left: 80px;  
    width: 100px;  
    height: 100px;  
    background: blue;  
    z-index: 2;  
}
```

z-index decides **which element appears on top**.

8. What are the different positioning properties in CSS?

There are **5 main position values**:

1. Static

Default position.

position: static;

2. Relative

Moves relative to its **normal position**.

position: relative;

top: 20px;

3. Absolute

Positioned relative to its nearest positioned ancestor.

position: absolute;

top: 10px;

4. Fixed

Stays fixed on the screen even when scrolling.

position: fixed;

bottom: 10px;

5. Sticky

Acts like relative until a certain scroll position, then behaves like fixed.

position: sticky;

top: 0;

Easy way to remember:

S R A F S

Static → Relative → Absolute → Fixed → Sticky

9. What is ECMAScript?

ECMAScript is a standard/specification that defines how scripting languages such as JavaScript should work.

Simple understanding:

ECMAScript

↓

Standard / Rules

↓

JavaScript follows these rules

For example, features such as:

- `let`
- `const`
- arrow functions
- classes
- promises

have been standardized through ECMAScript editions.

10. Difference between `var`, `let` and `const` + Temporal Dead Zone

`var`

```
var x = 10;
```

```
x = 20;
```

- Function scoped
- Can be redeclared
- Can be reassigned
- Hoisted and initialized with undefined

`let`

```
let x = 10;
```

```
x = 20;
```

- Block scoped
- Cannot be redeclared in the same scope
- Can be reassigned
- Has Temporal Dead Zone

`const`

```
const x = 10;
```

- Block scoped
- Cannot be redeclared
- Cannot be reassigned
- Has Temporal Dead Zone

Easy table:

Feature	var	let	const
Scope	Function	Block	Block
Redeclare	✓	✗	✗
Reassign	✓	✓	✗
TDZ	✗	✓	✓

What is Temporal Dead Zone?

TDZ is the period between entering a block and the point where a let or const variable is declared, during which accessing that variable causes a ReferenceError.

Example:

```
console.log(x);
```

```
let x = 10;
```

This gives an error because x is in the **Temporal Dead Zone** before its declaration.

Another example:

```
{
  // TDZ starts

  console.log(a); // Error

  let a = 10;

  // TDZ ends
}
```

Important difference with var:

```
console.log(a);
```

```
var a = 10;
```

Output:

undefined

But:

```
console.log(a);
```

```
let a = 10;
```

gives:

ReferenceError